

7c14c626cff0_9. Policy Gradient (2)

정책 경사

강의 9 – Policy Gradient (2)

문형식

시각 컴퓨팅 및 AI 연구실

고려대학교

Lecture credit: Emma Brunskill

1. Temporal Structure 를 사용한 Policy Gradient 수정
2. 기준값을 사용한 Policy Gradient 수정
3. 부트스트래핑을 사용한 Policy Gradient 수정

- 편향되지 않지만 매우 잡음이 많음
- 실용적으로 만들 수 있는 수정 사항
- Temporal Structure
- 기준값
- 몬테카를로 대신 부트스트래핑 사용
-
-

왜 'Temporal Structure'를 도입하는가?

우리는 해당 보상과 인과적 연결고리가 없는 미래 행동으로부터 발생하는 "노이즈"를 제거할 수 있다.

• 이전에:

• 동일한 논리를 반복하여 단일 보상 항 r_t '에 대한 경사 추정기를 유도할 수 있습니다.

• 인과성 제약: 현재 행동은 과거 보상에 영향을 줄 수 없으므로 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 는 $r_{t'}$ 에서 스케일링되어서는 안 됩니다. 여기서 $t' < t$.

$$\nabla_{\theta} \mathbb{E}[r_{t'}] = \mathbb{E} \left[r_{t'} \sum_{t=0}^{t'} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

•

• 무관한 잡음 제거: 단일 보상 항 r_t '에 대한 경사에 초점을 맞추으로써 해당 보상과 인과적 연결이 없는 미래 행동에서 발생하는 "잡음"을 제거할 수 있습니다.

• 예를 들어, $r_0 \log \pi(a_{100} | s_{100})$ 을 무시합니다.

• 이전:

$$\nabla_{\theta} \mathbb{E}_{\tau}[R] = \mathbb{E}_{\tau} \left[\left(\sum_{t=0}^{T-1} r_t \right) \left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \right]$$

$$\nabla_{\theta} \mathbb{E}[r_{t'}] = \mathbb{E} \left[r_{t'} \sum_{t=0}^{t'} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Formula over the whole...

$$\begin{aligned} \nabla_{\theta} V(\theta) &= \nabla_{\theta} \mathbb{E}[R] = \mathbb{E} \left[\sum_{t'=0}^{T-1} r_{t'} \sum_{t=0}^{t'} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^{T-1} r_{t'} \right] \end{aligned}$$

- 이 공식을 t 에 대해 합산하면

$$\begin{aligned}\nabla_{\theta} V(\theta) &= \nabla_{\theta} \mathbb{E}[R] = \mathbb{E} \left[\sum_{t'=0}^{T-1} r_{t'} \sum_{t=0}^{t'} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t, s_t) \sum_{t'=t}^{T-1} r_{t'} \right]\end{aligned}$$

- 보상-투-고 (G_t): 시간 t 부터 시작하여 궤적의 끝까지 얻은 보상의 합.
- 인과성을 고려한 Policy Gradient: 정책 파라미터는 점수 함수 $\nabla_{\theta} \log \pi_{\theta}$ 에 비례하여 업데이트된다.
- 분산 감소: 현재 행동이 영향을 미칠 수 없는 과거 보상을 버림으로써 경사 추정의 노이즈를 크게 줄인다.

Recall for a particular trajectory $\tau^{(i)}$, $\sum_{t'=t}^{T-1} r_{t'}^{(i)}$ is the return $G_t^{(i)}$

$$\nabla_{\theta} \mathbb{E}[R] \approx (1/m) \sum_{i=1}^m \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t, s_t) G_t^{(i)}$$

몬테카를로 정책 경사 (REINFORCE)

우도비 / 점수 함수와 시간적 구조를 활용합니다

$$\Delta \theta_t = \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) G_t$$

REINFORCE:

Initialize 정책 파라미터 θ arbitrarily

for each episode $\{s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}\} \sim \pi_{\theta}$

do for $t = 1$ to $T - 1$ do

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) G_t$

endfor

```
endfor  
return  $\theta$ 
```

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3), 229-256.

2. 기준값을 사용한 PG 수정

-
-

기준값을 도입하는 이유는 무엇인가요?

- 분산 감소: 정책 경사 추정기의 높은 분산을 낮추어 더 안정적이고 빠른 수렴을 이끌어내는 것이 주요 목표입니다.
- 상대적 성능 평가: 절대 보상 값에서 상대적 우위로 초점을 전환하여 평균 기대치보다 더 나은 경우에만 행동을 강화합니다.

기준값 도입의 목적 (b) (s)

-
- 균형 잡힌 경사: 학습 신호를 중심으로 하여 모든 행동이 상대적 품질과 무관하게 확률 증가를 일으키는 "모든 양의 보상" 문제를 방지합니다.

Policy Gradient: Introduce 기준값

- 해석: 행동의 로그 확률을 해당 행동이 기대 수익보다 얼마나 더 나은지에 비례하여 증가시킴
- 기준값 $b(s)$ 를 도입하여 분산을 감소시킴
- 어떤 기준값을 선택하더라도 경사 추정기는 편향되지 않음
-
- 거의 최적의 선택은 기대 수익임
-
-

기준값 $b(s)$ 는 편향을 도입하지 않는다-유도

(확률: 합이 1)

(상수의 미분은 0)

$$\begin{aligned}
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[\mathbb{E}_{s_{(t+1):T}, a_{t:(T-1)}} [\nabla_{\theta} \log \pi(a_t | s_t; \theta) b(s_t)] \right] \text{ (break up expectation)} \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \mathbb{E}_{s_{(t+1):T}, a_{t:(T-1)}} [\nabla_{\theta} \log \pi(a_t | s_t; \theta)] \right] \text{ (pull baseline term out)} \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \mathbb{E}_{a_t} [\nabla_{\theta} \log \pi(a_t | s_t; \theta)] \right] \text{ (remove irrelevant variables)} \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \sum_a \pi_{\theta}(a_t | s_t) \frac{\nabla_{\theta} \pi(a_t | s_t; \theta)}{\pi_{\theta}(a_t | s_t)} \right] \text{ (likelihood ratio)} \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \sum_a \nabla_{\theta} \pi(a_t | s_t; \theta) \right] \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \nabla_{\theta} \sum_a \pi(a_t | s_t; \theta) \right] \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} [b(s_t) \nabla_{\theta} 1] \text{ (probability: sum to 1)} \\
 &= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} [b(s_t) \cdot 0] = 0 \text{ (derivative of constant is zero)}
 \end{aligned}$$

기준값 $b(s)$ Does Not Introduce Bias-Derivation

- Policy Gradient Theorem
- $\nabla J \approx \mathbb{E} \nabla \log \pi \cdot G$

- $\nabla J \approx \mathbb{E} \nabla \log \pi \cdot G - b = \mathbb{E} \nabla \log \pi \cdot G - \mathbb{E} \nabla \log \pi \cdot b$

Original Direction Zero (0)

- Same Direction: Since $\mathbb{E} \nabla \log \pi \cdot b = 0$, the overall gradient direction (Expectation) remains identical to the original objective.
- Lower Magnitude: Subtracting b centers the learning signal around zero, reducing the 절대 크기 of the updates.
- Stability: Smaller, centered updates mean the policy doesn't "over-correct" or swing wildly, leading to much faster and more reliable convergence.

-

"Vanilla" Policy Gradient Algorithm

강의 8

From Lecture 8

$$\begin{aligned} \nabla_{\theta} V(\theta) &\approx \hat{g} = (1/m) \sum_{i=1}^m R(\tau^{(i)}) \nabla_{\theta} \log P(\tau^{(i)}; \theta) \\ &= (1/m) \sum_{i=1}^m R(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \end{aligned}$$

Initialize policy parameter θ , baseline b
for iteration=1, 2, ... **do**
 Collect a set of trajectories by executing the current policy
 At each timestep t in each trajectory τ^i , compute
 Return $G_t^i = \sum_{t'=t}^{T-1} r_{t'}^i$, and
 Advantage estimate $\hat{A}_t^i = G_t^i - b(s_t)$.
 Re-fit the baseline, by minimizing $\sum_i \sum_t ||b(s_t) - G_t^i||^2$,
 Update the policy, using a policy gradient estimate $\hat{g}$,
 Which is a sum of terms $\nabla_{\theta} \log \pi(a_t | s_t, \theta) \hat{A}_t$.
 (Plug $\hat{g}$ into SGD or ADAM)

endfor

From Lecture 8

$$\nabla_{\theta} V(\theta) \approx \hat{g} = (1/m) \sum_{i=1}^m R(\tau^{(i)}) \nabla_{\theta} \log P(\tau^{(i)}; \theta)$$

"Vanilla" Policy Gradient Algorithm

- Two-Network Architecture: 현대 정책 경사 방법은

- 일반적으로 두 개의 별도 함수 근사자
- (종종 두 개의 서로 다른 신경망)를 유지합니다.
- Policy Network ($\pi\theta$): 수익을 최대화하기 위한 최적의 행동 전략을 학습합니다.
- 가치 네트워크 ($V\phi$ or b): 현재 상태의 기대 수익을 추정하는 방법을 학습하여 안정적인 기준값 역할을 합니다.
- The Birth of 액터-크리틱: 이 구조는 액터-크리틱 방법의 근본적인 선구자이며, 여기서 "크리틱"
- (기준값)은 "액터"(정책)를 평가합니다.

기준값 선택: 가치 함수

- Q-함수 / 상태-행동-가치 함수를 상기하십시오:
- 상태 가치 함수는 훌륭한 기준값으로 사용될 수 있습니다

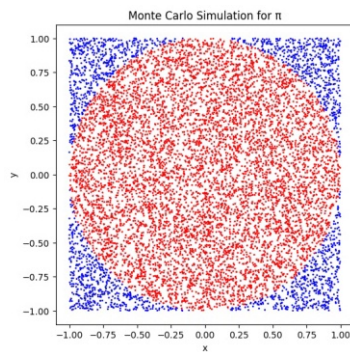
3. 부트스트래핑을 이용한 PG 수정

• Policy Gradient:

- Fixes that improve simplest estimator
- Temporal Structure (shown in above equation)
- 기준값 (shown in above equation)
- 부트스트래핑 instead of 몬테카를로
-

몬테카를로 (MC) 샘플링

- 정의: 수치 결과를 얻기 위해
- 반복적인 무작위 샘플링을 사용하는
- 광범위한 계산 알고리즘 클래스.
- 핵심 원리: 원칙적으로 결정론적이거나
- 해석적으로 풀기 너무 복잡한 문제를
- 해결하기 위해 무작위성을 사용한다.
- 큰 수의 법칙: 무작위 샘플 수가
- 증가함에 따라 결과의 평균이
- 진정한 기대값에 수렴한다.



MC vs. TD

- 실제 수익(G_t)을 여러 완료된 궤적(T)에서 평균하여 값을 추정하는 방법.
 - 학습은 종료 상태에 도달한 후에만 발생한다(T).
 - 실제 경험에 엄격하게 의존하지만 누적 잡음으로 인해 높은 분산을 겪는다.
- Temporal Difference (TD)
 - 최종 결과를 기다리지 않고 다른 학습된 추정치를 기반으로 가치 추정치를 업데이트하는 방법(부트스트래핑).
 - t 에서 $rt + \gamma V_{st+1}$ 을 사용하여 매 시간 단계마다 온라인으로 학습한다.
 - 초기 학습 단계에서 편향을 도입하지만 훨씬 더 안정적이고 효율적이다.

• 몬테카를로 (MC)

-

목표 선택

- 딜레마: 우리는 잡음이 있지만 진실한 목표 (G_t)와 안정적이지만 잠재적으로 "거짓말"하는 목표(부트스트래핑) 사이에서 선택해야 한다.
- 몬테카를로(MC): 높은 분산, 제로 편향. 안전하지만 수렴이 고통스럽게 느리다.

-

- 상태 시간 단계에서의 실제 보상 합계.
- MC 목표: $G_t = r_t + r_{t+1} + r_{t+2} + \dots$
- 부트스트래핑(TD): 낮은 분산, 높은 편향. 빠르고 안정적이지만 근사가 좋지 않으면 잘못된 해로 수렴할 위험이 있다.

- 단일 시간 단계의 실제 보상과 함수 근사 합계

- TD 목표: $r_t + \gamma V_{s_{t+1}}$

-

-

-

- 함수 근사: 신경망을 사용하여 값을 추정하는 것은 자연스럽게 편향을 도입하지만 큰 상태 공간을 처리하는 데 필수적이다.

-

액터-크리틱 방법

- V /Q 의 추정은 크리틱에 의해 수행된다
- 액터-크리틱 방법은 정책과 가치 함수의 명시적 표현을 유지하며 둘 다 업데이트한다
- A3C (Mnih et al. ICML 2016)는 매우 인기 있는 액터-크리틱 방법이다
-

Policy Gradient Formulas with Value Functions

- Recall:

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{\tau}[R] &= \mathbb{E}_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t; \theta) \left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t) \right) \right] \\ \nabla_{\theta} \mathbb{E}_{\tau}[R] &\approx \mathbb{E}_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t; \theta) (Q(s_t, a_t; \mathbf{w}) - b(s_t)) \right]\end{aligned}$$

- Letting the $\hat{V}$ be an estimate of the value V , we can represent the gradient in terms of the state-action advantage function

$$\nabla_{\theta} \mathbb{E}_{\tau}[R] \approx \mathbb{E}_{\tau} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(a_t | s_t; \theta) \hat{A}^{\pi}(s_t, a_t) \right]$$

where the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$

- From Returns to Estimates: 경험적 수익(G_t)에서 학습된 가치 함수(Q 및 V)로의 전환.
- 액터-크리틱 Foundation: 원시 수익을 학습된 Q -함수로 대체하여 액터-크리틱 프레임워크로 이동한다.
- Generalized Advantage: s, a 를 사용하여 ∇A^{π} 를 표현함으로써 부트스트래핑을 이용한 보다 유연하고 안정적인 업데이트를 가능하게 한다.

•

목표 선택: N-단계 추정기

비평가가 TD와 MC 추정기 사이의 임의의 혼합을 선택하여
실제 상태-행동 가치 함수를 대체할 수 있음을 유의하라.

$$tR^{(1)} = r_t + \gamma V(s_{t+1})$$

$$tR^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$$

$$tR^{(\infty)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

...

위의 식에서 기준값을 빼면 우도비 추정기를 얻는다.

$$tA^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$tA^{(\infty)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots - V(s_t)$$

$$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t^{(\infty)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots - V(s_t)$$

정책 경사 방법은 *Temporal Structure*를 활용하여 궤적에 대한 기대 수익을 추정합니다. REINFORCE는 우도비를 사용하여 경사 추정기를 계산하는 반면, 액터-크리틱 방법은 기준값을 사용하여 분산을 감소시킵니다. 부트스트래핑은 TD와 같은 방법을 사용하여 궤적을 부분적으로 샘플링함으로써 인과성 제약을 만족시킵니다. 몬테카를로 방법은 전체 궤적을 샘플링하여 큰 수의 법칙에 따라 기대 수익을 추정합니다. 신경망은 정책 파라미터와 가치 네트워크를 동시에 학습하여 과잉 보정을 방지하고, Q-함수와 상태 가치 함수를 통해 절대 크기를 조절합니다. 수치 결과는 실제 수익과 비교하여 모델의 성능을 평가합니다.

$$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t^{(\infty)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots - V(s_t)$$

"Vanilla" Policy Gradient Algorithm

```
Initialize policy parameter  $\theta$ , baseline  $b$ 
for iteration=1, 2, ... do
  Collect a set of trajectories by executing the current policy
  At each timestep  $t$  in each trajectory  $\tau^i$ , compute
    Advantage estimate  $\hat{A}_{it}^n$ 
  Update the policy, using a policy gradient estimate  $\hat{g}$ ,
    Which is a sum of terms  $\nabla_{\theta} \log \pi(a_t | s_t, \theta) \hat{A}_{it}^n$ .
    (Plug  $\hat{g}$  into SGD or ADAM)
endfor
```

참고, 우리는 어떤 혼합 추정기 $\hat{A}^n$ 을 사용할지 선택할 수 있습니다

정책 기반 강화학습의 현재 이점 요약

장점:

- 더 나은 수렴 특성
- 고차원 또는 연속 행동 공간에서 효과적
- 확률적 정책을 학습할 수 있음

•

단점:

•

- 일반적으로 전역 최적이지 아닌 지역 최적에 수렴
- 정책 평가가 비효율적이고 분산이 높음

(though 기준값 and Temporal Structure helps)

•